

```

In [18]: sentence = nltk.word_tokenize('Anne hoped she had outlived the age of blushing; but the age of emotion she certainly had not.')
In [19]: print(nltk.pos_tag(sentence))
[('Anne', 'NNP'), ('hoped', 'VBD'), ('she', 'PRP'), ('had', 'VBD'), ('outlived', 'VBN'), ('the', 'DT'), ('age', 'NN'), ('of', 'IN'), ('blushing', 'NN'), (';', 'P'), ('but', 'CC'), ('the', 'DT'), ('age', 'NN'), ('of', 'IN'), ('emotion', 'NN'), ('she', 'PRP'), ('certainly', 'RB'), ('had', 'VBD'), ('not', 'RB'), ('.', '.')]
In [20]:

```

What it returns is a list which pairs each word with its part of speech. Running the above you should have seen:

```
[('Anne', 'NNP'), ('hoped', 'VBD'), ('she', 'PRP'), ('had', 'VBD'), ...]
```

The above is the truncated output, you should see tags for all the words in the sentence, including punctuation.

The codes here, **NNP**, **VBD**, and **PRP** stand for “proper singular noun”, “past tense verb” and “personal pronoun” respectively. You can see all the tags and what they stand for using the command: `nltk.help.upenn_tagset()`. This is for the UPENN tagset, there are others available.

It's important to note that the parts of speech identification of a word depend on its context. In the above sentence, “blushing” is tagged as a noun, however the same word would be tagged as a verb in “He was blushing with embarrassment.”

You can see how useful it would be to take a sentence and get its nouns, verbs and adjectives. By knowing the parts of speech involved in a sentence, you can quickly determine the subjects, objects and actions being mentioned.

```

In [20]: chunker = nltk.RegexpParser('EXAMPLE: (<JJ>+<NN/?>*)')
In [21]: words = nltk.word_tokenize('I envy long golden luscious hair.')
In [22]: tagged = nltk.pos_tag(words)
In [23]: chunked = chunker.parse(tagged)
In [24]: print(chunked)
$
1/PRP
envy/VBP
(EXAMPLE long/JJ golden/JJ luscious/JJ hair/NN)
(/.)
In [25]:

```

NLTK also includes ready-made code for finding chunks that represent named entities, such as the names of people, places, organisations etc. You can try it on a tagged sentence using `nltk.ne_chunk(tagged)`.

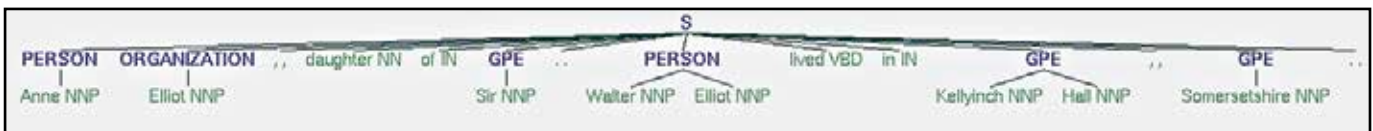
There are many more places where such tagged text can be further processed to enhance the kinds of operations you perform on text.

Without any training or additional data, `ne_chunk` has managed to do a decent job of detecting named entities.

Next Steps

What we've covered till now is the very basics of what NLTK is capable of, and even then we looked at NLTK in isolation without considering other useful libraries.

A huge part of natural language processing is actually learning from data, developing a model, and using it to better classify text



Chunking

Once you have a tagged sentence, NLTK includes many other tools that operate on it to perform more useful tasks. One such tool is chunking, which allows you to use an extended regex syntax to search for patterns in POS tagged text. For instance you could search for and tag a series of adjectives followed by a noun, which would tag the sequence “long golden luscious hair” in a sentence.

Let's see this in action:

```

chunker = nltk.RegexpParser('EXAMPLE: (<JJ>+<NN/?>*)')
words = nltk.word_tokenize('I envy long golden luscious hair.')
tagged = nltk.pos_tag(words)
chunked = chunker.parse(tagged)
print(chunked)

```

The first line here initialises our chunker to look for sequences of adjectives followed by the noun they apply to. “JJ” here is code for adjectives, and the “+” in regex means look for one or more. All kinds of nouns start with “NN”, so “NN.?” means “NN” possibly followed by another character. The text “EXAMPLE” just tells it to tag such sequences as “EXAMPLE”, you could call it anything.

You will see that the output contains this line:

```
(EXAMPLE long/JJ golden/JJ final/JJ luscious/JJ hair/NN)
```

Many operations in NLTK produce Tree objects which can be viewed graphically by calling their `draw` method.

This program has tagged this sequence from the sentence. You can also try `chunked.draw()` to see this as a tree graph.

in the way we want. A good example of that is Sentiment Analysis. Sentiment Analysis software can predict whether a given text conveys a positive, neutral or negative tone. In order for it to work properly, you need to train the software by feeding it examples of positive and negative texts. The more examples you give it, the better the result.

While NLTK gives you access to tonnes of existing datasets, sometimes you might want to create your own. This is especially true if the text you want to work with is very specific to a domain.

We have also glossed over the powerful WordNet module. WordNet is a huge collection of words that have been linked together by similarity in meaning (like a thesaurus) but also other relations such as antonyms, hypernyms and hyponyms. It is also a dictionary, and includes meanings and example usages of words.

NLTK also gives you access to a large volume of text in multiple languages (including some Indian languages) to play with. It's also incredibly easy to import your own text.

Finally, NLTK isn't everything. There are a number of other packages for Python and other programming languages that focus on different areas of NLP. NLTK is a good place to explore first since it is broad in scope, yet easy to understand and has good documentation and a large number of tutorials from reliable sources available online. >

You can find out more about NLTK at nltk.org and read a free book about NLTK at nltk.org/book/.